

HAND GESTURE CONTROL

A Local Real-Time Two-Hand Tracking and Gesture-to-Action
Desktop Application

Technical Project Proposal

Author	Amirsam Eftekharinia
Version	1.0
Date	September 2026
Document type	Technical / Engineering Proposal
Primary domain	Computer Vision · HCI · Desktop Systems
Implementation status	Functional prototype with substantial test coverage

This document describes the architecture, implementation, algorithms, limitations, and future directions of the Hand Gesture Control system. All technical claims are grounded in the current source code and test suite. Planned or unimplemented functionality is explicitly labeled.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Motivation
5. System Overview
6. System Architecture
7. Technical Implementation
8. Technologies and Tools
9. Key Technical Features
10. Algorithmic and Technical Methodology
11. Data Flow
12. Security, Privacy and Reliability
13. Testing and Validation
14. Performance and Resource Considerations
15. Current Status
16. Limitations
17. Future Work
18. Use Cases
19. Comparison with Existing Approaches
20. Technical Contribution
21. Reproducibility
22. Project Structure
23. Development Roadmap
24. Conclusion
25. References

Executive Summary

Hand Gesture Control is a desktop application that enables users to control system functions—volume, media playback, screenshots, application-specific toggles, and optional keyboard shortcuts—through hand gestures observed by a webcam. All perception and decision-making occur on-device; no network service is required for core operation.

The system uses MediaPipe's Hand Landmarker (Tasks API, VIDEO mode) to obtain 21 landmarks per hand for up to two hands, derives scale- and largely rotation-independent geometric features, classifies finger states and static gestures, supports user-recorded custom pose templates, and dispatches confirmed gestures to a configurable action layer. A real-time pipeline processes only the newest available frame, never a backlog, and is isolated from the UI thread through queued signals and explicit cross-thread handoff primitives.

The implementation is structured as a layered Python application (camera, vision backends, tracking, gestures, actions, pipeline, UI, config) with a substantial automated test suite, including regression tests for previously fixed concurrency and path-resolution defects. A Windows packaging path via PyInstaller is provided. The project is best characterized as a solid engineering system for local gesture-driven HCI rather than a novel research algorithm contribution; its value lies in careful integration, robustness under real desktop constraints, and extensibility.

1. Introduction

Human–computer interaction on desktop systems remains dominated by keyboard and mouse (or trackpad). Gesture-based control has been explored extensively in research and in commercial products (depth cameras, dedicated hardware, VR/AR controllers), but practical, always-available control from an ordinary webcam is still uncommon outside specialized applications.

Commodity RGB webcams and modern on-device hand landmark models (notably MediaPipe) make continuous hand pose estimation feasible on typical laptops. The remaining engineering problem is not merely detection: it is building a complete, responsive, configurable application that (1) maintains stable hand identity across frames, (2) classifies poses robustly under rotation and scale change, (3) avoids accidental activation, (4) never freezes the UI, (5) respects privacy by keeping video local, and (6) can be extended without rewriting the pipeline.

This project addresses that engineering problem. It does not introduce a new landmark model or training pipeline; it builds a production-minded application layer around an existing detector, with explicit attention to threading, configuration, packaging, and testability.

2. Problem Statement

Controlling common desktop functions without touching input devices is useful when hands are occupied, dirty, or distant from the keyboard—for example during presentations, media consumption, or accessibility-oriented use. Existing options include:

- Dedicated hardware (depth sensors, gesture controllers) — cost and availability barriers.
- Cloud-based vision APIs — latency, cost, and privacy concerns for continuous camera streams.
- Ad-hoc scripts or demos that draw MediaPipe landmarks but lack stable tracking, temporal confirmation, action mapping, or a maintainable architecture.

Technical constraints that any practical solution must confront include: variable lighting and camera quality; hand rotation that breaks naive axis-aligned finger rules; brief occlusion; the need to distinguish intentional gestures from transitional poses; the requirement that the UI remain responsive while vision runs at interactive rates; and the risk that powerful system actions (volume, media keys, hotkeys) fire accidentally.

The problem this project solves is therefore: **design and implement a local, configurable, two-hand gesture control application for commodity webcams that is robust enough for daily use and structured enough to extend.**

3. Objectives

3.1 Primary Objectives

- Real-time detection and tracking of up to two hands from a webcam stream.
- Classification of a set of static gestures with temporal confirmation and edge-triggered events.
- Configurable mapping from gestures to system and application actions.
- Fully local processing with no cloud dependency for core features.
- A usable desktop UI with live preview, settings, and gesture mapping controls.

3.2 Secondary Objectives

- Support for user-recorded custom pose templates without code changes.
- Continuous pinch-based volume control.
- Snap (dynamic) gesture detection.
- Windows packaging path that produces a standalone distribution.

3.3 Engineering Objectives

- Clear separation of camera I/O, detection backends, tracking, gesture logic, actions, and UI.
- Cross-thread safety: no direct GUI access from worker threads; synchronized config apply.
- Newest-frame-only pipeline policy (drop frames rather than queue latency).
- Path resolution independent of process working directory (source and frozen builds).
- Automated tests that do not require a physical camera or GPU.

3.4 Research / Experimental Objectives

The project is primarily an engineering system. Experimental goals supported by the architecture (but not claimed as completed research results) include evaluating pose-only custom gestures for user-defined control, and providing a clean backend interface so alternative detectors can be compared under the same tracking and gesture stack. No formal user study or published benchmark suite is included in the current repository.

4. Motivation

Technical. Modern hand landmark models deliver usable keypoints on CPU or modest GPU at interactive rates. The interesting work is downstream: geometry that survives rotation, temporal logic that reduces false triggers, identity tracking across misses, and an application architecture that keeps vision off the UI thread without losing configurability.

Practical. A single binary or a simple Python install that turns a laptop camera into a gesture controller is more accessible than specialized hardware for many users. Local-only operation avoids the privacy and latency costs of continuous video upload.

Engineering. Many vision demos stop at “draw the skeleton.” Shipping something that survives packaging, path quirks, COM threading on Windows, camera disconnects, and user configuration is a different class of problem. This project treats those concerns as first-class requirements.

5. System Overview

At a high level the system is a closed-loop desktop application:

Webcam frames → **Camera manager (capture thread)** → **Frame pipeline (worker thread)** → detection backend → multi-hand tracker → geometry & finger state → gesture engine → action dispatcher → **overlay + UI updates (GUI thread)**.

User configuration (thresholds, mappings, custom templates, camera index, model path) is loaded from a per-user config directory, validated against a typed schema, and can be updated at runtime through mutex-guarded pending state applied by the worker between frames.

Input Webcam / OpenCV	Perception MediaPipe + geometry + finger state	Recognition Static / custom / pinch / snap + temporal confirm	Action Registry + system adapters	UI PySide6 preview, settings, mapping
---------------------------------	--	---	---	---

Figure 1. Logical pipeline stages from camera input to UI output.

6. System Architecture

The codebase is organized into packages under `app/`. Each package has a narrow responsibility.

6.1 Major Components

Component	Responsibility	Thread / context
camera/	Capture, device enumeration, reconnect with bounded backoff	Dedicated QThread
vision/	Backend interface, MediaPipe & mock, landmarks, geometry, finger state, smoothing	Pipeline worker
tracking/	Stable hand IDs across frames; motion history; miss tolerance	Pipeline worker
gestures/	Static classification, custom templates, pinch, snap, gesture engine	Pipeline worker
actions/	Action registry, mapping, system volume/media/screenshot/hotkeys	Worker + background executor
pipeline/	Frame orchestration; newest-frame only; pending config apply	Dedicated QThread
ui/	Theme, widgets, camera view, settings, mapping panel, overlays	GUI thread
config/	Schema, defaults, load/save, custom gesture store	GUI + validated apply
utils/	Paths, logging, perf monitor, GuiInvoker, BackgroundExecutor	Shared utilities
main_window.py	Integration: owns camera + pipeline, wires ActionContext	GUI thread

6.2 Threading Model

Three concurrent domains exist: (1) camera capture thread, (2) pipeline worker thread, (3) Qt GUI thread. Camera and pipeline workers run hand-rolled loops (not `QThread.exec()`) so they control frame timing. Configuration changes use mutex-guarded pending values applied by the worker between frames. GUI updates always use Qt queued connections or `GuiInvoker`. Blocking OS calls run on a `BackgroundExecutor`.

6.3 Backend Abstraction

Detection is behind a small interface with MediaPipe and mock implementations. The mock backend enables deterministic tests without a camera. GPU delegate is attempted first for MediaPipe and falls back to CPU; the failure is cached for the process lifetime.

7. Technical Implementation

7.1 Core Components

Entry point `main.py` configures high-DPI policy, application font, icon, logging, and launches `MainWindow`. Version string `0.1.0` is defined and logged at startup. Settings and custom gestures persist as JSON under the platform user config directory.

7.2 Vision and Geometry

MediaPipe produces 21 landmarks per hand. Preferring world landmarks when available, geometry code builds a palm-local frame, palm size as scale reference, per-finger straightness ratios, joint angles, and normalized thumb-index distance. Finger state maps continuous geometry to discrete labels (extended / folded / partial / unknown), with dedicated thumb logic using abduction angle.

7.3 Gesture Engine

Static gestures are classified from finger states and geometry thresholds. Custom gestures store averaged curl ratios, thumb abduction, and pinch distance; matching uses a weighted Euclidean distance, independent of handedness. The engine applies hold-time confirmation and cooldown so events are edge-triggered. Pinch supports continuous volume mapping; snap is a short temporal detector.

7.4 Actions and System Integration

An action registry maps action IDs to handlers. System adapters cover volume (Windows virtual-key events plus optional pycaw read-back; Linux pactl/amixer; macOS osascript), media keys, screenshots, and optional pyautogui hotkeys. Most disruptive mappings ship disabled in defaults. Windows COM usage is confined to the action worker thread with per-thread initialization.

7.5 Configuration, Logging, Errors

Configuration uses a typed schema with clamping/validation and a schema version field. Corrupt JSON is backed up and replaced with safe defaults. Logging writes rotating files under the user config directory; UI surfaces short user-facing messages via typed AppError subclasses.

7.6 Packaging

build_windows.bat creates a clean venv, installs dependencies, downloads the model if missing, generates the icon, and runs PyInstaller with an onedir layout. The spec collects MediaPipe and OpenCV fully and trims unused Qt modules. End-to-end verification on physical Windows hardware is still incomplete.

8. Technologies and Tools

Technology	Purpose	Role in system
Python 3.10+	Primary language	Entire application
MediaPipe Tasks	Hand Landmarker model	21 landmarks × up to 2 hands, VIDEO mode
OpenCV	Camera I/O, image ops, screenshots	Capture & display path
NumPy / SciPy	Numeric geometry	Vectors, angles, distances, smoothing
PySide6 (Qt 6)	Desktop UI	Main window, widgets, signals/slots
pyautogui (optional)	Hotkeys / media keys	Non-Windows fallback actions
pycaw / comtypes (Win)	Volume read-back	Optional absolute volume sync
pytest	Automated tests	Unit + regression suite (dev dependency)
PyInstaller	Windows packaging	Onedir executable distribution
Pillow (build-time)	Icon generation	Multi-resolution .ico export

No cloud APIs, databases, or training frameworks are used at runtime. The MediaPipe model is downloaded once from Google's published storage URL.

9. Key Technical Features

Feature	Purpose	Implementation notes
Two-hand tracking	Stable identity under brief occlusion	Proximity + handedness matching; miss budget
Rotation-aware geometry	Finger state under varied poses	Palm-local frame; ratios & angles
Temporal gesture confirmation	Reduce accidental triggers	Hold duration + cooldown; edge events
Custom pose templates	User-defined gestures without code	Curl / angle / pinch feature distance
Pinch volume	Continuous analog control	Normalized thumb–index distance → volume
Newest-frame pipeline	Bound latency under load	Overwrite latest frame; drop backlog
Backend interface	Swap detectors	MediaPipe + mock; factory selection

Feature	Purpose	Implementation notes
Safe defaults	Avoid surprise system actions	Most mappings disabled until enabled
Launch-independent paths	Correct model/icon resolution	Project root or PyInstaller bundle

10. Algorithmic and Technical Methodology

10.1 Landmark to Geometry

Given 21 landmarks (image-space always; world-space when provided), define palm center and a scale measure (palm size). Construct orthonormal axes from palm geometry. For each non-thumb finger, straightness is derived from the ratio of tip-to-MCP distance to the fully extended chain length and joint angles. Thumb state additionally uses abduction angle relative to the palm frame. Normalized thumb-index distance is Euclidean distance divided by palm size, yielding a scale-independent pinch feature.

10.2 Static Gesture Classification

A rule-based classifier maps the discrete finger-state set and selected geometry thresholds to gesture IDs (fist, open_palm, pointing, thumb_up, peace, ok, pinch, three_fingers, four_fingers). Rules are ordered so more specific poses take precedence. Confidence is derived from continuous measures.

10.3 Custom Gesture Matching

A template stores mean finger curl ratios (five values), thumb abduction angle, and normalized pinch distance. Live poses are sampled into the same feature space. Distance is a weighted Euclidean combination after scaling angle and pinch terms. Matching is hand-agnostic by construction.

10.4 Temporal Engine

Raw per-frame labels are not acted on immediately. The engine requires a pose to remain stable for a configurable confirmation time, then emits a single edge event and applies a cooldown.

10.5 Multi-Hand Association

Detections are matched to existing tracks by spatial proximity and handedness. Tracks survive up to a configured number of missed frames. Position history supports velocity estimates for dynamic detectors.

10.6 MediaPipe Timestamp Discipline

VIDEO mode requires strictly monotonically increasing timestamps. The backend generates timestamps from a monotonic clock under a lock so duplicate frames or reconnects cannot violate the invariant.

11. Data Flow

End-to-end path for a single processed frame:

1. Camera thread captures BGR frame + timestamp → stores as “latest” under mutex.
2. Pipeline worker reads latest frame (may skip if none new).
3. Optional landmark smoother updates temporal filters per hand track.
4. Vision backend → FrameResult (0–2 HandObservations with landmarks).
5. Multi-hand tracker assigns/updates track IDs; drops invalid observations.
6. Geometry + finger-state computed per hand.
7. Gesture engine produces zero or more confirmed GestureEvents.
8. Action dispatcher maps events; blocking work queued to BackgroundExecutor.
9. Overlay renderer draws skeleton/status onto a frame copy.
10. Results emitted via Qt signal to MainWindow (GUI thread).
11. Camera view, status widgets, gesture history, toasts update.

Configuration changes write pending config applied between frames. Custom gesture templates are stored separately from main settings JSON.

12. Security, Privacy and Reliability

12.1 Privacy

Camera frames are processed entirely on-device. The application does not upload video or landmarks. The only intentional network use is the optional one-time model download. A persistent “Camera active” indicator is shown while the camera is live.

12.2 Action Surface and Defaults

Gesture-triggered actions can change volume, send media keys, take screenshots, or emit keyboard shortcuts. Default mappings enable only a small subset; others ship disabled until the user enables them.

12.3 pyautogui Failsafe

When pyautogui is available, its corner-mouse failsafe is set to False. This is intentional for hands-free sessions but removes a common emergency stop.

12.4 Trust Boundaries and Hardening

There is no remote attack surface in the core design. Local risks include corrupt config files (mitigated by validation and backup-on-corrupt) and the inherent power of enabled system actions. Subprocess volume calls use fixed argument lists without shell=True. This document does not claim absolute security.

13. Testing and Validation

Automated tests use pytest with QT_QPA_PLATFORM=offscreen. Tests rely on synthetic landmarks and the mock backend; they do not require a physical camera, GPU, or downloaded model.

Area	Examples of coverage
Geometry & finger state	Straightness, angles, thumb rules, classification
Static gestures	Rule outcomes for canonical poses
Custom gestures & store	Template build, match distance, JSON load/save, corrupt handling
Smoothing / pinch / snap	Filter behavior, continuous mapping, temporal snap logic
Multi-hand tracker	Association, miss tolerance, identity continuity
Gesture engine	Confirmation, cooldown, event edges
Actions & config	Registry dispatch, schema validation, defaults
Hardening regressions	Timestamp monotonicity, path resolution, cross-thread config, volume threading

Hardening regressions encode bugs found in a prior audit so they cannot silently return. Not present: formal user studies, published GPU benchmarks, or hardware-in-the-loop Windows volume verification.

14. Performance and Resource Considerations

No formal published benchmark numbers are claimed. scripts/benchmark.py measures on-machine frame time and detection latency using the same PerfMonitor the app uses at runtime. Results are machine-dependent.

Design choices affecting performance: newest-frame-only policy; MediaPipe GPU with CPU fallback; smoothing and tracking that avoid identity resets on single-frame glitches; packaged size dominated by MediaPipe and OpenCV native libraries (hundreds of MB). Typical expectation: laptop with webcam and modern CPU.

15. Current Status

Capability	Status
Two-hand landmark detection (MediaPipe)	Implemented
Mock backend for tests / offline UI	Implemented
Geometry, finger state, static gestures	Implemented
Custom pose recording and matching	Implemented
Pinch volume, snap detection	Implemented
Action mapping UI + system adapters	Implemented
Per-user config + custom gesture store	Implemented
Cross-thread pipeline & GUI safety patterns	Implemented
Automated unit/regression tests	Implemented
Model download script	Implemented
Windows PyInstaller packaging scripts	Implemented (HW verification incomplete)
Motion / trajectory custom gestures	Planned
Two-hand relational gestures (zoom/rotate)	Planned
Additional detector backends (e.g. ONNX)	Planned / interface ready
Formal user study or published accuracy metrics	Not present

16. Limitations

- **Detector dependency.** Accuracy inherits from MediaPipe Hand Landmarker; this project does not train a model.
- **Pose-centric gestures.** Custom gestures are pose templates, not full motion trajectories.
- **Handedness flicker.** Brief left/right flips can disrupt track identity.
- **Volume read-back.** Setting works cross-platform; reliable get is best-effort on Linux.
- **Camera names.** Enumeration uses generic labels; no portable friendly device names via OpenCV alone.
- **Windows packaging verification.** Scripts exist; full HW validation is not claimed complete.
- **No formal accuracy dataset.** Tests use synthetic landmarks only.
- **Distribution size.** Packaged builds are large due to native CV/ML libraries.
- **pyautogui failsafe disabled.** Emergency mouse-corner abort is intentionally off.

17. Future Work

17.1 Short-Term

- Complete Windows hardware verification for packaging and volume control.
- Expand regression coverage around camera reconnect and multi-camera switching.
- Optional CI workflow running the offline pytest suite.

17.2 Medium-Term

- Motion-based custom gestures (trajectory / DTW-style matching).
- Improved track association under handedness flicker.
- Interaction zones and pointer/air-click experimental mode.

17.3 Long-Term

- Two-hand relational gestures (pinch-zoom, rotate, scale).
- Second real detector backend (e.g. ONNX) under the same abstraction.
- Session recording with baked-in overlays.
- Accessibility-focused evaluation with external users (study design not yet defined).

18. Use Cases

Scenario	User	Problem	How the system helps
Media control while hands occupied	General user	Cannot reach keyboard	Pinch volume / media gestures
Presentation support	Presenter	Need discrete control at distance	Mapped gestures for media or hotkeys
Accessibility experimentation	Limited fine motor use of mouse	Alternative input channel	Large stable poses as commands
On-device HCI demo	Engineer / student	Need complete stack, not a notebook	Runnable app + tests + packaging

These are plausible applications consistent with implemented features, not claims of deployed production users or clinical validation.

19. Comparison with Existing Approaches

Aspect	This project	Typical MediaPipe demo	Dedicated depth hardware
Input	RGB webcam	RGB webcam	Depth / structured light
Tracking stack	Custom multi-hand + geometry	Often draw landmarks only	Vendor SDK
Actions	Configurable system actions	Usually none	Varies by product
Privacy	Local only	Usually local	Usually local
Architecture	Layered, testable modules	Often single script	Closed binary
Custom gestures	User pose templates	Rare	Product-specific
Cost	Software + existing camera	Software	Hardware purchase

Comparison is qualitative. No claim is made that this project outperforms commercial gesture systems in accuracy or latency.

20. Technical Contribution

This project should not be described as a fundamental algorithmic breakthrough. Its contribution is engineering and systems integration:

- A coherent layered architecture separating detection, tracking, recognition, actions, and UI.
- Explicit cross-thread design suitable for Qt + realtime vision workers.
- Geometry and gesture logic that prefer normalized, pose-stable features over brittle pixel heuristics.
- User-extensible custom poses without modifying source.
- Packaging and path-resolution discipline aimed at real desktop launches.
- A regression suite that encodes concrete production defects.

If research novelty is required, it would need additional evaluation (datasets, user studies, comparative metrics) that are not in the current repository.

21. Reproducibility

Another developer can reproduce the software environment as follows:

1. Clone the repository; use the project root (contains main.py).
2. Create a virtual environment with Python 3.10+.
3. pip install -r requirements.txt
4. python scripts/download_models.py (internet required once).
5. python main.py — grant camera permission when prompted.
6. Tests: pip install -r requirements-dev.txt && pytest

7. Windows build (on Windows): run build_windows.bat.

User-specific settings live outside the repo. The mock backend and synthetic fixtures allow logic tests without hardware.

22. Project Structure

```
HandGestureControl/
■■■■ app/                # Application package
■   ■■■■ actions/        # Registry, mapping, system actions
■   ■■■■ camera/         # Capture & device management
■   ■■■■ config/         # Schema, defaults, persistence
■   ■■■■ gestures/       # Static, custom, pinch, snap, engine
■   ■■■■ pipeline/       # Real-time frame worker
■   ■■■■ tracking/       # Multi-hand identity
■   ■■■■ ui/             # PySide6 UI & widgets
■   ■■■■ utils/          # Paths, logging, threading helpers
■   ■■■■ vision/         # Backends, geometry, finger state
■   ■■■■ main_window.py  # Integration / main window
■■■■ assets/icon/        # Application icons
■■■■ assets/models/      # Model download target (not committed)
■■■■ configs/            # Documentation of config locations
■■■■ scripts/            # download_models, benchmark, generate_icon
■■■■ tests/              # pytest suite + fixtures
■■■■ main.py             # Entry point
■■■■ requirements.txt     # Runtime dependencies
■■■■ requirements-dev.txt # Test dependencies
■■■■ requirements-build.txt
■■■■ HandGestureControl.spec
■■■■ build_windows.bat
■■■■ LICENSE             # MIT
■■■■ README.md
```

23. Development Roadmap

Phase	Focus	Outcomes
Phase 1 (done)	Core pipeline + UI + tests	Runnable local app, offline tests, packaging scripts
Phase 2	Hardening & platform verification	Confirmed Windows builds; expanded reconnect tests
Phase 3	Richer interaction	Motion templates; improved tracking under flicker
Phase 4	Extensibility & evaluation	Second backend; optional user study design

Phases after Phase 1 are directional, not scheduled commitments.

24. Conclusion

Hand Gesture Control is a local desktop system that turns commodity webcam input into configurable, temporally confirmed hand-gesture commands. It combines MediaPipe-based landmark detection with custom geometry, tracking, gesture logic, and a Qt application structured for responsiveness and extensibility. The implementation prioritizes engineering concerns that demos often omit: threading discipline, safe defaults for system actions, path resolution under packaging, and automated regression tests.

The project's limitations are equally clear: it inherits detector accuracy from MediaPipe, focuses on pose rather than full motion templates, lacks formal in-the-wild accuracy studies, and still needs complete Windows hardware verification for packaging. Within those bounds, it is a coherent, reproducible technical foundation for on-device gesture-driven interaction and for further experimental extensions.

25. References

[1] Google MediaPipe. Hand Landmarker documentation.
https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker

-
- [2] C. Lugaresi et al. MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172, 2019.
 - [3] Qt Company. Qt 6 / PySide6 Documentation. <https://doc.qt.io/qtforpython/>
 - [4] OpenCV Team. OpenCV Library Documentation. <https://docs.opencv.org/>
 - [5] PyInstaller Development Team. PyInstaller Documentation. <https://pyinstaller.org/>
 - [6] Python Software Foundation. Python 3.10+ Documentation. <https://docs.python.org/3/>

End of Technical Project Proposal — Hand Gesture Control v1.0 — September 2026